Fusion 360 MCP

Model parts and assemblies in Autodesk Fusion 360 by talking to Claude.

You describe what you want — "a 60×40×20 mm enclosure, 2 mm walls, four M3 tapped holes in the corners" — and Claude builds it in Fusion, looks at the result with a screenshot, checks the dimensions, and fixes its own mistakes. It can also export STEP/STL/DXF and save versioned designs.

This is a hobbyist project shared as-is. It works well for the bread-and-butter of mechanical CAD (boxes, cylinders, holes, fillets, shells, tapped holes, multi-part assemblies with joints) and hands off to raw Fusion API code for the exotic stuff.

What you get

About 60 tools Claude can call, grouped as:

- **Primitives & features** — boxes, cylinders, holes, fillets, shells, tapped holes (with correct tap-drill sizing for hand-tapping). Cutting tools auto-report volume change and warn on no-op cuts.
- **Parametric** — named user parameters so models stay adjustable.
- **Faces** — sketch and drill on a selected face, not just the base plane.
- **Assemblies** — components, occurrences, linear copies of hardware, grounding, and rigid / revolute joints.
- **Verification & diagnostics** — return a screenshot to Claude, measure real distances, ASCII cross-sections (solid/void scans), face/edge listings with stable IDs, and a rich one-call model-state report.
- **Async & resilience** — long operations run as pollable jobs with incremental progress; liveness/state stay responsive even during a hung op; the listener refuses known host-crashing API calls.
- **Document safety** — every state read shows the active document name + lineage; saves report which lineage they wrote; save-as-new and read-only version listing guard against overwriting the wrong design.
- **Export & save** — STEP, STL (3D print), DXF (laser/waterjet), and save / open / list designs across sessions.
- **Escape hatch** — `run_fusion_code` / `run_fusion_code_safe` run arbitrary Fusion API Python (with injected unit/observability helpers and documented sketch-plane mapping), the latter with automatic rollback on error.

Full list: `docs/TOOLS.md`.

How it works (the 30-second version)



Two long-running pieces:

1. **MCPListener** — a small add-in that runs *inside Fusion* and executes modeling commands on Fusion's main thread.
2. **fusion_mcp_server.py** — an MCP server that **Claude Desktop launches for you** in the background; it relays Claude's tool calls to the listener.

You don't start the server yourself — Claude Desktop does, based on its config. You just keep Fusion open with the add-in running.

Quick start

1. **Install** — follow `docs/INSTALL.md`. It covers the add-in, the Python deps, and the Claude Desktop config, with fixes for the common snags.
2. **Check your setup** — run the preflight: - Windows: double-click `launch.bat` - macOS/Linux: `./launch.sh` It tells you exactly what's missing.
3. **Confirm it's live** — in Claude Desktop, ask: **"Ping Fusion."** A reply of "alive" means everything's connected.
4. **Build something** — try:

> "Make a 20 mm cube and show me."

then the real shakedown:

> "Make a 60×40×20 mm parametric box, shell it to 2 mm with an open top, add an M3 tapped hole 5 mm in from each top corner, and show me top and iso views."

Requirements

- Autodesk Fusion 360 (current version).
- Claude Desktop (the desktop app — this uses local MCP over stdio).
- Python 3.10+ (3.10–3.12 are the safest; see INSTALL note on 3.13/3.14).

Files in this package

File	What it is
`MCPListener.py`	The add-in that runs **inside Fusion**.
`fusion_mcp_server.py`	The MCP server **Claude Desktop launches**.
`launch.py` / `launch.sh` / `launch.bat`	Preflight health-checker.
`docs/claude_project_instructions.md`	Paste into a Claude Project to teach it the conventions.
`requirements.txt`	Python dependencies.
`docs/INSTALL.md`	Step-by-step install + troubleshooting.
`docs/pdf/`	Print-ready PDFs of every doc (plus `ALL-DOCS` combined).
`docs/render_pdfs.py`	Regenerates those PDFs after you edit the markdown.
`docs/TOOLS.md`	Reference for every tool.
`docs/ARCHITECTURE.md`	How the pieces fit, for the curious / for hacking.
`LICENSE`	MIT.

A few honest caveats

- The listener executes arbitrary Python sent to **localhost only** (`127.0.0.1:8765`). That's required for it to work, but don't expose that port to a network. See ARCHITECTURE for the security model.
- The canned tools assume principal planes and box-like geometry for face selection. Complex/organic shapes use the raw-code escape hatch — Claude knows when to switch.
- This isn't an official Autodesk or Anthropic product. It's glue code. Treat early runs as a shakedown and keep your work saved.

Happy modeling.

Changelog

1.8.1

Two regressions found by stress-testing 1.7.0/1.8.0 rather than assuming they were sound. Both were introduced by the optimization work itself.

Fixed - `run_batch` could fire a destructive operation during collection

`run_batch` used a DENYLIST of non-batchable tools, so anything not listed ran for real during what should have been a dry collection pass. Verified: batching `clear_document` actually WIPED the document and then reported the batch as failed - data loss plus a misleading error.

Now fail-safe: - An explicit **allowlist** (27 geometry/parameter tools). A tool omitted simply cannot be batched, instead of silently executing. - Every operation is validated **before** collection mode is entered, so an invalid batch has no side effect at all. - `_send_async` refuses outright while collecting, so the async path cannot fire. - A reentrant lock around collection, so a concurrent call cannot be swallowed into the batch buffer and silently never executed.

Fixed - terse mode hid the body of report tools

The 1.8.0 compaction dropped indented continuation lines, but the entire output of `document_report` is indented - so the tool built specifically to prevent wrong-document overwrites was reduced to its first line. `get_state`'s open-document list, `list_faces`, `cross_section` boundaries and others were affected the same way.

Compaction is now opt-in per call: 18 report tools pass `compact=False`, because their body IS the detail. Action tools still compact on clean success, so the token saving is retained where it was actually free.

1.8.0

The remaining measured optimizations from the 1.7.0 profiling pass.

Claude tokens

- **Terse-on-success output (`set_verbosity`).** Detail is valuable exactly when something is WRONG; on a clean success it is pure token cost paid on every call. Indented explanatory lines are now hidden when the payload contains no trouble marker (***, WARNING, ERROR, FAILED, UNCONFIRMED,

MISMATCH, REFUSED, Traceback, no-op, STALE). Any one of those disables compaction for the whole payload, so failures always print in full. `set_verbosity('full')` restores everything.

Host compute

- **Face-iteration caps.** `get_model_state` computed the area of every face of every body to count slivers, and `list_faces` measured every face before capping the output - each area/centroid read is a kernel query. Both now bound at 4000 faces and say so, instead of stalling on a dense model.
- `cross_section` **refines boundaries by bisection** instead of scanning uniformly. A uniform scan wastes queries deep inside solid regions and still only resolves boundaries to span/N. Now: coarse pass, then ~8 bisections per transition. Measured on a hollow box with walls at 10.700/49.300 mm: reported 10.699/49.301 mm from 47 kernel queries, versus 1.94 mm precision from the old 31-query uniform scan - roughly 1900x better precision for 16 extra queries. Also reports span widths and the query count.

Round trips

- `job_status` **documents polling backoff** (~5s, 10s, 20s, then 30s). Each poll is a round trip that returns the whole accumulated stdout again; get_state is cheaper when only liveness is needed.

1.7.0

Optimization pass - measured, not guessed.

Host CPU (a regression introduced in 1.6.0)

- **The identity refresh was a 4-second heartbeat.** It queued a Fusion main-thread job every 4 seconds for as long as Fusion was open (~21,600 interrupts/day), each enumerating every open document and touching dataFile - which raises, expensively, on builds where app.data is broken. Cost was paid continuously even when nobody was using the tool. It is now LAZY: /state requests a refresh only when its cache is actually stale, coalesced to at most one in flight and one per 2 s. Idle cost is now zero.

Speed - round trips

- `run_batch(operations_json)` fuses N tool calls into ONE round trip and ONE Fusion job. Every tool call otherwise costs a full hop (model -> server -> HTTP -> main thread -> back), so ten holes cost ten hops. Each operation still prints its own output and per-step volume deltas; stop_on_error aborts at the first failure. Verified: a 5-operation batch used 1 round trip instead of 5. Not for steps whose arguments depend on an earlier step's result.

Claude tokens

- **Tool schemas are sent on EVERY request.** Measured at ~5,488 tokens. Seven of the largest docstrings were trimmed to their imperative warnings, with the reference material moved to a new on-demand `fusion_help(topic)` tool (topics: helpers, planes, traps, workflow). Now ~4,677 tokens - a saving of **~811 tokens on every request**, roughly 40k over a 50-turn session.
- `view_model` **resolution is now a parameter, defaulting to 800x600 instead of 1200x900.** An image costs about (w*h)/750 tokens, so the default drops from ~1,440 to ~640 - ~800 tokens saved per screenshot*, the single biggest per-call saving. Raise it only when judging fine detail; 640x480 (~410 tokens) suffices for a quick sanity look.

1.6.1

Audit for other instances of the bug class behind 1.6.0 (implicit target / unverified success). Found four more, all real, all fixed.

- **40 tools assumed** `app.activeProduct` **IS a Design.** It is not, in the Drawing, CAM, Render or Simulation workspace - `.rootComponent` then fails obscurely or targets something unexpected. All 40 now route through an injected `_require_design()` that raises a clear error naming the actual product type and document, and telling the user to switch workspace.
- **Exports announced success without checking.** `em.execute()` 's return value was discarded and the tool printed "exported ..." regardless - the same "thinks it saved but it didn't" failure applied to files. STEP/STL/DXF now verify the return value AND that the file exists with non-zero size, printing CONFIRMED with the byte count or an explicit failure.
- `set_parameter` **never read back.** Fusion silently rejects or clamps an expression it does not accept, so a parameter could report as set to 50 mm while holding something else. It now reads the stored expression back and reports CONFIRMED or MISMATCH with both values.
- **Joints reported success without verifying.** `joints.add()` returning without error does not mean a joint exists. Now checks the joint count increased.
- `clear_document` **- the most destructive tool - checked nothing.** It now prints the target document name, lineage, and current contents before deleting anything.

1.6.0

Document identity. Claude could not reliably tell which document was open, which produced the worst failures in the project: overwriting an unrelated design, starting a new document instead of continuing in the open one, and reporting a save that never happened. Four compounding root causes, all fixed.

- `doc.dataFile` **routes through** `app.data` **.** When app.data is dead (common on this build, survives a Fusion restart), lineage read as missing - and the old logic treated "no lineage" as "never saved", so it called `saveAs` and created a DUPLICATE of a document that was already saved. `save_document` now tries the NON-DESTRUCTIVE `doc.save()` first whenever lineage is unreadable, and refuses to

fall back to `saveAs` without explicit `force_new_lineage=True`. Verified: in that scenario it now makes zero saveAs calls.

- **Identity was a stale cache.** `get_state` only refreshed after a job ran, so switching documents in the Fusion UI left it reporting the previous one. The listener now refreshes identity on a timer, and `/state` reports its own snapshot age and a `fresh` flag.
- `isSaved` **was the sole branch signal.** Every identity signal (`isSaved`, `isModified`, `dataFile`, version, folder) is now read INDEPENDENTLY and reported side by side, so a contradiction is visible instead of collapsed.
- **No visibility of open documents.** `document_report()` is a live, authoritative read listing EVERY open document and which is active, ending in an explicit verdict that distinguishes "never saved" from "cannot tell". `activate_document(name_or_index)` binds work to the user's document.
- **Saves are now verified, not assumed.** `save_document` reports version before/after and prints CONFIRMED only when an independent signal agrees (version bump / modified cleared / lineage appeared); otherwise UNCONFIRMED, with instructions not to report the document as saved.

1.5.0

Round-2 field report (organic modelling: spline surfaces of revolution, lofted bifurcation, swept elbows, ~15 fillet campaigns).

Added - edge selection & fillet verification (the biggest gap)

- `select_edges(...)` - predicate selection by adjacent **surface-type pair**, z range, and length, returning tempIds. `which=all/vertical/top` could not express what every real fillet needs, so the good search logic in `fillet_edges_smart` was locked behind a selector that couldn't reach it - the same ~30-line predicate was hand-rolled about fifteen times in one session.
- `list_edges` **now reports the surface-type pair** (e.g. `Cylinder|Cone`), so callers stop re-implementing the classifier.
- `fillet_edges_smart` **accepts** `edge_tempids` and now verifies **topology as well as volume**: it reports which faces disappeared and warns when a face smaller than `4r^2` was consumed. Round 1's failure was a fillet that ADDED geometry far away; this session's was the inverse - a 2.0 mm fillet that ate a 1.3 mm rim annulus and rounded the tube ends away while the volume delta (-18 mm^3 vs ~28 expected) passed every threshold.
- `fillet_preview(...)` - shows the affected faces and flags any smaller than `4r^2` BEFORE committing.
- **A zero volume delta is now reported as a RESULT, not a failure** - it means the surfaces already meet tangentially and need no fillet.

Added - escape hatches

- `force_delete_feature(name)` - resolves a MUTUAL-REFERENCE DEADLOCK where two features each blame the other and neither suppress nor `deleteMe()` works in any order. Rolls the marker back before deleting (what the Fusion UI does), then reports what else broke. This turned a blocked session into a 2-minute fix.
- `sweep_tube_check(...)` - measures a path's minimum curvature radius against the tube radius. `ASM_SELF_INTER` blames the profile/taper, but on a circular profile the PATH is the cause.
- `respline(...)` - rebuild a fitted spline with new fit points (Fusion can move but not add them). States honestly that downstream references break, and reports which features became unhealthy.

Fixed

- **Degenerate-edge crash.** Tangentially-meeting lofts leave zero-length edges with <2 faces; `e.faces.item(1)` raised `Bad index parameter`. Guards are now inside the listener helpers (`edge_ok`, `safe_edges`, `edge_face_pair`) and every edge-walking tool, so callers stop rediscovering it via traceback.
- `explain_fillet_error` **knew almost none of the codes.** Added `ASM_ABL_NO_GEOM_EF`, `ASM_ABL_CVXTY_VIOLATION`, `ASM_BL_NO_VTX_GEOM`, `ASM_BL_CHG_PT_COMPLEX_VERT`, `ASM_BL_CANNOT_REORDER`, `ASM_INCONS_VERT_ATT`, `ASM_SELF_INTER`, `FILLET_NO_EDGE_FOUND`, `EDGE_REFERENCE_LOST`.
- `refresh_geometry()` - `computeAll()` does not guarantee a fresh `body.volume`; identical readings after a sketch edit nearly led to concluding a correct edit had failed.
- `path_report(path)` - segment count + length, so a mis-chained path (which silently produces a short sweep) is visible.

Notes on "did not land" items

- **clear_document (item 5):** real, and fixed in 1.4.1 - the 1.4.0 rewrite was correct but kept a fallback that still bulk-deleted bodies, the exact hang path.
- **YZ docstring (item 7):** corrected in 1.4.0. The docstring now carries its own version stamp, because Claude Desktop caches tool schemas at startup - a fixed docstring can still *display* stale without a full restart.
- **app.data (item 6):** `save_document`'s fallback works and is verified. `open_document` / `list_recent_designs` genuinely cannot be worked around; this is now stated plainly rather than implied.
- `ping_fusion` **reports both server and listener versions** and warns when they differ (added 1.4.1) - a stale *server* previously produced no mismatch signal.

1.4.1

Follow-up to two field reports that turned out to be **version mismatch** - both symptoms were v1.2.1 behaviour, and both were already fixed in 1.4.0. The real finding was that nothing made that diagnosable, plus one genuine remaining hazard.

- `clear_document` **fallback sweep hardened.** The 1.4.0 rewrite used reverse-order `entity.deleteMe()` correctly, but kept a fallback that still bulk-deleted `bRepBodies` - the exact path that hung for 10+ minutes on a parametric document. That sweep now runs ONLY when there is no timeline (direct-modelling docs). On a parametric doc with leftovers it reports what remains and points at `retire_body()` instead of grinding. Verified: zero bulk deletes on a parametric doc.
- `ping_fusion` **now reports BOTH server and listener versions** and warns when they differ. The existing protocol handshake only caught a stale *listener*; a stale *server* (this case) produced no mismatch signal at all, so an old docstring and an old code path looked like live bugs.

1.4.0

Fixes from a full modelling session (splitter funnel). Several of these were silent-wrong-geometry bugs, which are the expensive kind.

Fixed (P0 - wrong facts that produced wrong geometry)

- **The documented YZ sketch-plane mapping was WRONG.** Docs claimed `sketch X -> world Y, sketch Y -> world Z`; measured behaviour is `sketch X -> world -Z, sketch Y -> world +Y`. Building off the wrong mapping yields a body with a correct bounding box and correct profile area that only reveals itself several operations later. Docstring corrected, and new injected helpers `P_xy/P_xz/P_yz` (world mm in, correct sketch point out) plus `verify_plane_mapping(sketch)` mean nobody has to reason about it.
- `report_volume_delta` **cried wolf on every Join.** It summed ALL bodies, so a Join that merges two bodies is volume-neutral and got flagged as a no-op - training you to ignore the one warning that matters. Now takes a `capture_state(target_body)` snapshot and warns only when body count AND volume are both unchanged.
- **Nothing detected silent geometry destruction by fillets.** A tangent-chained 1.5 mm fillet propagated ~20 mm and reverted a bore radius, reporting success with a valid body and no slivers. `fillet_edges` now checks the volume delta against `~0.1*r^2*L` and warns on an order-of-magnitude overshoot; `fillet_volume_sanity()` exposes the same check to raw code.

Fixed (P1 - broken tools)

- `export_stl` **was unusable at every quality setting.** The enum names `*QualityTriangleMeshRefinement` do not exist on this build; the real members are

`MeshRefinementLow/Medium/High/Custom`. Now uses those, with a graceful fallback that prints the available members if they change again.

- **`app.data` failures killed save/list/open for a whole session.** `save_document` now detects `InternalValidationError`, falls back to `doc.save()` for an already-saved document (the path that does work), and says clearly that a new lineage needs the Fusion UI. `list_recent_designs` explains the failure instead of returning a bare error.
- **`clear_document` hung for 10+ minutes and took Fusion down.** Reimplemented as reverse-order `item(i).entity.deleteMe()` (TimelineObject has no `deleteMe`) with `progress()` per feature, and it now runs through the async job system by default so the channel is never blocked.

Fixed (P2)

- **Bodies added via `BaseFeature` return a stale reference** - `ALL_TOOL_BODY_REFERENCE_LOST` when reused as a Combine tool. `cut_prisms` and `commit_tool_body()` now re-fetch by name after `finishEdit()`. This was a real bug in code shipped in 1.3.0.

Improved (P3 - observability)

- **`view_model`**: added `bottom`; an unknown orientation is now an ERROR instead of silently returning the previous view; added `zoom_x/y/z_mm` + `zoom_radius_mm` to frame a small detail instead of always fitting the whole model.
- **`cross_section`**: added `at_a_mm` / `at_b_mm` to pin the held axes, so you can scan through a specific feature rather than the bbox mid-plane.
- **`fillet_edges_smart`** - searches tangent-chain on/off x descending radii, verifies volume delta, rolls back overshoots, reports what worked. **`explain_fillet_error`** translates raw ACIS/ASM codes into fixes.
- **`smallest_faces(body)`** prints the actual smallest face areas; `sliver_face_count` threshold raised to 0.05 mm^2 (0.010-0.014 mm^2 faces are visible artifacts that the old cutoff missed).
- **Transport timeouts** now return a structured RUNNING status naming the job id, with an explicit "do not re-issue" note - a timed-out call had already completed a save in one case.
- **`retire_body()`** + docs for the replace-a-body idiom (`removeFeatures.add(old)`), which is strictly safer than clearing.

Docs

- New "API traps on this build" section in TOOLS.md covering the plane mapping table, `TimelineObject.entity`, stale BaseFeature handles, `is`-comparison on sketch entities, `addByCenterStartSweep` normalisation, `setMidRadii` list types, `app.data` failures, and silent fillet propagation.

1.3.0

Security fix plus the curved-surface toolset the docs already promised.

Fixed (P0)

- **SECURITY: argument injection.** Tool arguments were pasted directly into Python source text, so a filepath or name containing a quote broke out of the literal and executed arbitrary code inside Fusion. All ~22 interpolation sites now emit proper Python literals via `repr()`. Verified by executing generated code for 9 tools x 7 adversarial payloads (63 runs) with an injection canary: zero escapes.
- **P0: Windows paths raised SyntaxError.** Same root cause - `C:\parts\x.step` was pasted raw, so backslashes became escape sequences. Fixed by the same change; paths with backslashes, trailing separators and apostrophes now work.

Added (P1)

- `wrap_cut` - wrap a flat sketch onto a non-planar face and cut/emboss to depth. Previously referenced by the docs but MISSING from the build, so following the instructions led straight into a wall. Uses the correct `EmbossFeatures.createInput(profiles, faces, depth)` signature (negative depth cuts), cuts per-profile with individual volume checks, and reports ok/no-op/failed counts with sector guidance.
- `project_to_surface`, `construction_plane_offset`, `construction_plane_tangent`, `sketch_axis_map` - the supporting cast the wrap workflow needs. The construction-plane tools replace `ConstructionPlaneInput.setByPlane`, which is environment-blocked here, and echo world position so unit slips surface immediately.
- `cut_prisms` + injected `tbm()` / `commit_tool_body()` helpers - many prism cuts in ONE timeline feature via `TemporaryBRepManager`. Depends on neither construction planes nor face tempIds, so it sidesteps the two most fragile things in this environment.

Docs

- TOOLS.md gains a curved-surface section and a TemporaryBRepManager section with a worked example.

1.2.1

Fixes from a project-wide review triggered by a real degraded-state incident (old listener + new server = every payload empty while status read OK).

- **CRITICAL: switched to ThreadingHTTPServer.** The v1.2.0 listener used a single-threaded HTTPServer, so while `/exec_sync` held the one HTTP thread waiting on a job (up to ~110s), `/ping` and `/state` could not answer - silently defeating the "separate always-responsive channel" design. Verified fixed: ping now answers instantly during a blocked sync call.

- **Version/protocol handshake.** The listener now reports `listener_version` + `protocol` in every response; the server checks it and returns a loud PROTOCOL MISMATCH message with exact fix steps when the add-in is stale - instead of silently rendering empty payloads as STATUS: OK. This makes the incident that prompted this release impossible to miss.
- **Fail-loud capture health.** Envelopes include `stdout_bytes`; the server marks ok-but-silent results with "(no output captured)".
- **Job-store pruning.** Finished jobs are capped at the 50 most recent so long sessions don't grow memory unboundedly.
- **Docs:** INSTALL now warns that Fusion loads the add-in from its AddIns folder, not from wherever the project files live - updating one does not update the other.

1.2.0

Async execution and document safety — driven by a second round of real-use feedback. The core fix: a 4-minute timeout that told you nothing about what actually happened in Fusion. Execution is now decoupled from the HTTP response.

Added — async job model (the big one)

- **Listener is now async.** `POST /exec` enqueues a job and returns a `job_id` immediately; the operation runs on the main thread and is polled via `GET /job/<id>`, which returns a structured envelope (`status`, `stdout`, `traceback`, `elapsed_ms`) and **partial stdout while still running** (incremental progress).
- **`run_async(code)`** + **`job_status(job_id)`** tools for long operations that shouldn't block. Plus a `progress(msg)` helper to mark stages visible mid-run.
- **Ping/state on a separate channel.** `GET /ping` and `GET /state` are served from cache without the main thread, so they STAY RESPONSIVE during (or after) a hung operation. A hung op and a dead listener are now distinguishable.
- **`get_state()`** — cheap, guaranteed-fast document identity + counts (no geometry work). The safe first call for crash recovery.
- **Structured result envelope, always.** Every execution returns status/stdout/traceback/elapsed; no more bare "no result received". The sync path returns partial output and a job id if the op outlives the wait.
- **Watchdog** flags jobs that exceed their budget as `timeout` (clear signal rather than a dead channel).

Added — document safety

- **Document identity surfaced everywhere.** `get_state` and `get_model_state` lead with the active document **name + lineage id**, preventing wrong-document overwrites.

- `save_document(name, create_new=False)` now reports **which lineage it versioned**, and `create_new=True` saves a brand-new lineage so a new part never buries an unrelated design's history.
- `list_versions(design_id="")` — read-only enumeration of a lineage's saved versions, so a bad save is recoverable and the user can be pointed to the exact version to promote.
- `open_document` reports the new active lineage.

Added — documentation

- Sketch-plane / axis mapping (XY/XZ/YZ + offset planes, incl. the XZ→world-−Z inversion) documented in `run_fusion_code`'s docstring.

Notes

- `run_fusion_code` uses the bounded sync path (~100s) then hands off to polling; no behavior change for short ops.
- Volume-delta no-op detection and the `mm()` unit helper shipped in 1.1.0 and already cover two earlier feedback items.

1.1.0

Observability, diagnostics, and stability. Injected unit/observability helpers (`mm()`, `assert_scale()`, `report_volume_delta()`, etc.); auto volume-delta reporting with no-op warnings on cut/shell tools; `cross_section()`, `list_faces()`, `list_edges()`, `heartbeat()`; listener-level refusal of known host-crashing calls; richer `get_model_state()`.

1.0.0

First public release. Fusion 360 add-in listener + MCP server with ~30 tools (primitives, parameters, faces, assemblies/joints, measurement, visual feedback, export, persistence), preflight launcher, and full docs.

Install guide

Three parts to install, then a verification step. Budget ~15 minutes the first time. Read the **Common snags** at the bottom if anything misbehaves — every issue listed there is one a real user actually hit.

```
Part A — the Fusion add-in (the listener)
Part B — Python + dependencies
Part C — register with Claude Desktop
Verify  — preflight check, then ping from Claude
```

Part A — Install the Fusion add-in

The listener must be an **Add-In**, not a Script. (Scripts run once and exit; add-ins stay loaded — which the listener needs.) This trips people up, so follow the type carefully.

1. Open Fusion 360.
2. Press **Shift+S** (or **Utilities → Add-Ins → Scripts and Add-Ins**).
3. Click the **Add-Ins** tab (not Scripts).
4. Click the green **+** to create. In the dialog choose **Type: Add-In**, **Language: Python**, **Name:** `MCPListener`, then Create. - This creates a folder at:
   - **Windows:** `%appdata%\Autodesk\Autodesk Fusion 360\API\AddIns\MCPListener\`
   - **macOS:** `~/Library/Application Support/Autodesk/Autodesk Fusion 360/API/AddIns/MCPListener/`
5. Select `MCPListener` in the list, click the **pencil / Edit** to open its folder, and **replace the generated** `MCPListener.py` with the one from this package (same filename).
6. Back in the Add-Ins tab, select `MCPListener`, tick **Run on Startup**, and click **Run** (▶). A popup should say *"MCPListener running on http://127.0.0.1:8765"*.

> **The AddIns folder is the only copy Fusion runs.** If you keep the project files elsewhere (a repo/working folder), updating that copy does NOT update the add-in - you must copy the new `MCPListener.py` into the AddIns folder above and restart the add-in (Stop › Run). A stale add-in with a newer server now reports PROTOCOL MISMATCH instead of failing silently.
>
> Leave Fusion open while using Claude. With "Run on Startup" ticked, the listener auto-starts each time Fusion launches.

If you accidentally made it a Script (no "Run on Startup" toggle appears for it, and its path contains `\API\Scripts\`): delete/ignore that one and recreate it under the **Add-Ins** tab as in step 4. The file can't just be moved — add-ins and scripts are registered differently.

Part B — Python and dependencies

1. Have **Python 3.10+** installed and on your PATH. Check:
   `bash python --version # or: python3 --version`
2. Put this package folder somewhere stable, e.g. `C:\Users\<you>\fusion-mcp\` or `~/fusion-mcp/`.
3. Install dependencies **into the interpreter you'll point Claude at**. The safest way is the `-m pip` form so it can't land in the wrong Python: `bash python -m pip install -r requirements.txt` (or `python3 -m pip install -r requirements.txt`)

> **Why `-m pip`?** If you have several Pythons, a bare `pip install` can install into a different one than the launcher/Claude uses, and you'll get "No module named 'requests'" or "No module named 'mcp'" even though pip said success. `python -m pip` installs into *that* python, every time.

Part C — Register with Claude Desktop

1. Open **Claude Desktop → Settings → Developer → Edit Config**. **Always reach the file this way** — it opens the exact file Claude Desktop reads (`claude_desktop_config.json`). Don't go hunting in folders; the location varies and editing the wrong file is the #1 reason "nothing happens."
2. Add a `fusion360` server. If the file is empty, make it exactly:

Windows: `json { "mcpServers": { "fusion360": { "command": "python", "args": ["C:/Users/<you>/fusion-mcp/fusion_mcp_server.py"] } } }`

macOS / Linux: `json { "mcpServers": { "fusion360": { "command": "python3", "args": ["/Users/<you>/fusion-mcp/fusion_mcp_server.py"] } } }`

- Use an **absolute** path with **forward slashes** (or doubled `\\` on Windows — never single backslashes).
- If the config already has `mcpServers` with other entries, **add `fusion360` as another key inside it** — don't paste a second `mcpServers` block or a second top-level `{ }`. 3. **Fully quit and reopen** Claude Desktop (not just close the window).

Verify

1. With Fusion open (listener running) and Claude Desktop restarted, run the preflight from this folder: - **Windows:** double-click `launch.bat`, or `launch.bat check` - **macOS/Linux:** `./launch.sh` (run `chmod +x launch.sh` once first)

Everything should read **OK**. If `requests` or `mcp` shows FAIL, redo the `python -m pip install` with the right interpreter. 2. In Claude Desktop, confirm the `fusion360` tools appear (tools/connector icon near the chat box), then ask:

> **Ping Fusion.**

A reply of **"alive"** confirms the whole chain: Claude → server → listener. 3. First model:

> **Make a 20 mm cube and show me.**

Is the server running? (a common confusion)

You **don't** start `fusion_mcp_server.py` yourself, and the launcher does **not** keep it running. Here's the model:

- **Claude Desktop launches the server** automatically (as a hidden background child process) whenever it starts, based on the config. It stops when you quit Claude Desktop.
- **The launcher only inspects** your setup — it never leaves anything running. When `launch.bat` says "press any key" and the window closes, nothing was started or stopped.
- **To know it's running:** ask Claude to "Ping Fusion." If the `fusion360` tools show up and the ping works, it's running. If the tools never appear, Claude Desktop couldn't launch it — check the config path and the interpreter.

The only time you'd run the server in a terminal is debugging: `python launch.py serve` (used with the MCP Inspector). Not needed normally.

Common snags

"No module named 'requests'" (or 'mcp') even after pip installed it. You installed into a different Python than the launcher/Claude uses. Re-run with `python -m pip install -r requirements.txt` using the *same* command (`python` vs `python3`) that your Claude config's `command` uses. The preflight prints which interpreter it's using — match that.

Preflight says "Claude Desktop config not found." The launcher checks the standard location (`%APPDATA%\Claude\claude_desktop_config.json` / `~/Library/Application Support/Claude/...`).

If yours is elsewhere, that's fine — trust **Settings → Developer → Edit Config** as the source of truth instead. The WARN doesn't block anything.

Tools don't appear in Claude Desktop. Almost always the config: invalid JSON (a stray/missing comma), a relative or wrong path, or the wrong interpreter. Validate the JSON, use an absolute path, fully restart Claude Desktop. A quick JSON sanity check:

```
python -m json.tool < "path/to/claude_desktop_config.json"
```

If that errors, your JSON is malformed.

"Connection error" when a tool runs, but the server is fine. That's the *listener* — Fusion isn't open, or the add-in isn't running. Shift+S → Add-Ins → MCPListener → Run. Then "Ping Fusion" again.

Python 3.13 / 3.14. The SDK generally imports on very new Pythons, but they run ahead of what some dependencies expect. If the server misbehaves in ways the config can't explain, install a 3.11 or 3.12 alongside and point the Claude config `command` at that interpreter's full path. This is the first thing to try for weird, hard-to-place errors.

OneDrive "doubled" paths. Some Windows setups have paths like `C:\Users\<you>\Documents\Documents\...` (OneDrive redirection — the doubled `Documents` is real). Whatever your actual path is, make the Claude config match it character-for-character.

Geometry comes out 10× too big or small. A units slip somewhere in custom code. All tool arguments are millimeters; Fusion's API is centimeters. Ask Claude to verify with `get_model_state` or `measure_distance`.

Save / open / list-designs tools error. Those need you signed into Fusion with an **active project** selected in the Data Panel. They don't work offline or on a brand-new unsaved doc.

TOOLS.md — Fusion 360 MCP tool reference

Argument safety: all arguments are emitted as proper Python literals, so Windows paths (`C:\\parts\\x.step`), apostrophes, and quotes are handled correctly and cannot alter the code that runs inside Fusion.

Unit rule: every argument is in **millimeters**. Internally tools convert to Fusion's centimeters. When you (or Claude) write raw code, the API is in cm.

Face selection rule: `top/bottom` = max/min Z, `front/back` = min/max Y, `right/left` = max/min X. Chosen by extreme face-centroid position. Reliable on box-like parts; verify with `view_model` on complex geometry.

Core

`ping_fusion()`

Liveness + busy check, served on a **separate channel** from geometry execution — so it answers even while a long op runs on the main thread (a hung op no longer blocks ping). Reports `busy` and `current_job`.

`get_state()`

Cheap, guaranteed-fast doc state from the listener cache: active document name, lineage id, and body/sketch/component/timeline counts. Does **no geometry work** and is served without the main thread — safe to call during a heavy op or after a suspected hang. Use first for crash recovery. (For full per-body detail, `get_model_state`.)

`heartbeat()`

Crash-vs-busy check with explicit recovery instructions. Same liveness data as `ping_fusion`, plus guidance: if it errors, restart the add-in; if `busy=True`, poll `job_status`.

`run_async(code)`

Start a long operation **without waiting** — returns a `job_id` immediately. Poll `job_status(job_id)` for incremental progress and the final result. Use for anything expected to exceed ~90s (large patterns,

heavy booleans, full-assembly rebuilds). Same injected helpers and unit rules. Call `progress(msg)` in your code to mark stages visible while it runs.

`job_status(job_id)`

Structured result of an async job: `status` (queued/running/ok/error/timeout), `elapsed_ms`, stdout collected **so far** (works mid-run), and traceback on failure. Poll until status is ok/error/timeout.

`run_fusion_code(code)`

Execute arbitrary Python in Fusion via the **bounded sync path**: waits up to ~100s and returns a structured envelope (status / elapsed / stdout / traceback). If the job outlives the wait it keeps running and stays pollable — the response tells you the job id. `adsk`, `app`, `ui` in scope, plus injected helpers (below). **Use `mm(x)` for every length** instead of hand-dividing by 10.

The docstring also documents the **sketch-plane axis mapping** (XY/XZ/YZ and offset planes) so the XZ→world-−Z inversion doesn't cost you a debugging cycle.

Injected helpers (available in `run_fusion_code` and `run_fusion_code_safe`, no import needed): - `mm(x)` / `cm(x)` — mm→cm for values you pass to the API. - `to_mm(x)` / `to_mm2(a)` / `to_mm3(v)` — cm→mm for reporting. - `assert_scale(max_mm=2000)` — raise if any body is 10×-too-big (units slip). - `total_volume()` — summed cm³ of all bodies; capture **before** a boolean. - `report_volume_delta(before, label)` — print the change and **scream if ~0** (silent no-op boolean). - `capture_state(target_body)` / `report_volume_delta(before, label, target_body)` — the correct before/after pair. Tracks body count **and** the target body's own volume, so a volume-neutral **Join** is no longer flagged as a false no-op. - `fillet_volume_sanity(r_mm, edge_len_mm, delta_cm3)` — catches a fillet that propagated beyond its edges. - `sliver_face_count(body, min_area_mm2=0.05)` — threshold raised; prefer `smallest_faces(body, n=5)`, which prints the actual smallest areas. - `P_xy(x,y)` / `P_xz(x,z)` / `P_yz(y,z)` — world mm → correct sketch point. - `verify_plane_mapping(sketch)` — print a plane's measured local→world basis. - `retire_body(body)` — the safe replace-a-body idiom. - `edge_ok(e)` / `safe_edges(body)` / `edge_face_pair(e)` — degenerate-edge guards and the surface-type-pair classifier. - `path_report(path)` — segment count + total length, so a mis-chained path is visible (a short sweep is otherwise silent). - `refresh_geometry()` — force a recompute and report whether volumes moved. - `log_sketch_areas(sketch)` — print each profile area in mm² (catches scale slips). - `delete_face(face)` — delete one face the way Fusion accepts (`DeleteFaceFeatures`).

Recommended pattern for any cut/join in raw code:

```
_b = total_volume()
# ... do the boolean ...
report_volume_delta(_b, 'mycut'); assert_scale()
```

Known-unstable (refused by the listener): move-face via `defineAsFreeMove` (hard-crashes the host) and `defineAsOffset` (missing on this build). Use parametric edits / `offsetFeatures` / shell instead.

`run_fusion_code_safe(code)`

Transactional `run_fusion_code`: records the timeline marker and, on error, rolls back and deletes features created after it. Same injected helpers.

`clear_document()`

Delete all bodies, sketches, and timeline features. Returns design to empty.

Parameters

`set_parameter(name, value_mm)`

Create or update a user parameter (mm). Build parametric-first for anything that might change later.

`list_parameters()`

List user parameters and their expressions.

Primitives (root component, XY plane)

> **Auto-observability:** the material-removing tools (`drill_hole`, `hole_on_face`, `tapped_hole`, `shell_body`) automatically report the **volume delta** of the operation and **warn if it's ~zero** (a silent no-op cut that didn't touch the body). `shell_body` also reports sliver-face count. You no longer need to probe volume yourself to confirm a cut worked.

`create_box(length_mm, width_mm, height_mm, origin_x_mm=0, origin_y_mm=0)`

Rectangle on XY extruded up by height. Returns new body name.

`create_cylinder(diameter_mm, height_mm, origin_x_mm=0, origin_y_mm=0)`

Circle on XY extruded up.

`drill_hole(diameter_mm, x_mm, y_mm, depth_mm, body_index=0)`

Cut a hole straight down from XY at (x, y) to depth.

`fillet_edges(radius_mm, body_index=0, which="all")`

Round edges. `which`: `all` / `vertical` (vertical-running edges) / `top` (edges lying on the top face plane). Warns if nothing matched.

`shell_body(thickness_mm, body_index=0, open_top=True)`

Hollow to a wall thickness. `open_top=True` removes the top face (open box); `False` makes a closed hollow body.

`tapped_hole(thread_size, x_mm, y_mm, depth_mm, body_index=0, tap_drill_only=True)`

Tapped hole on the top (XY) plane, cutting down. `thread_size` e.g. `M3`, `M3x0.5`, `M4`, `M5`, `M6`, `1/4-20`, `10-32`, `#6-32`. - `tap_drill_only=True` (default): cuts only the correctly-sized tap-drill clearance hole — best for 3D-printed and hand-tapped parts. - `False`: also applies a **modeled** thread (visualization / CNC); best-effort, may be skipped with a warning if Fusion can't match the designation. Known tap-drill sizes include metric M1.6–M12 and common imperial; unknown sizes fall back to M3's drill — pass an explicit size you know is in the table, or ask to extend it.

Face-referenced features

`sketch_on_face(component_name="", which="top", body_index=0)`

Select a face by orientation and stash it. Empty `component_name` = root. Call `hole_on_face` immediately after (single-slot; don't interleave).

`hole_on_face(diameter_mm, u_mm, v_mm, depth_mm, through=False)`

Drill into the stashed face. `(u, v)` are mm offsets from the face centroid along the face's own axes. `through=True` cuts all the way.

Measurement & state

`get_model_state()`

Comprehensive design state in one call. Per body: name, validity, volume (cm³), bbox (mm), face/edge counts, and **sliver-face count** (flags a bad boolean). Plus body/sketch counts and the **timeline feature names**. Replaces hand-rolled introspection; makes crash recovery a one-call operation.

`measure_distance(which_a, which_b, body_index=0)`

True centroid-to-centroid distance (mm) between two oriented faces. Use to verify wall thickness, overall size, gaps — more trustworthy than the bbox.

`cross_section(body_index=0, axis="z", at_mm=None, span_axis="x", samples=41, at_a_mm=None, at_b_mm=None)`

Scan a line of points through a body and report solid/void as an **ASCII row** (`#`=solid, `.`=void) plus the **mm coordinates of every solid/void boundary**. The best non-visual diagnostic for trapped voids, missed cuts, and wall thickness. Holds the two non-span axes at their mid-value (or at `at_mm` along `axis`) and sweeps `span_axis` across the bbox. **Pin the held axes explicitly** with `at_a_mm` / `at_b_mm` to scan through a specific feature — e.g. a vent tube at x=3.8: `cross_section(span_axis='y', at_a_mm=3.8)`. Example: `cross_section(axis="z", at_mm=10, span_axis="x")` → reads off wall positions along X at Z=10 mm.

`fillet_edges_smart(radius_mm, body_index=0, which="all", min_radius_mm=0.2)`

Fillet that **searches** for a working configuration: for each radius (halving down to `min_radius_mm`) it tries tangent-chain on then off, verifies the volume delta against `~0.1*r^2*L`, rolls back an overshoot, and reports which combination worked. Exists because a blend that fails at every constant radius, every variable radius, and as a chamfer can succeed on the *first* radius once tangent chaining gives it a run-out.

`explain_fillet_error(message)`

Translates raw ACIS/ASM codes (`ASM_BL_END_TOO_CMPLX`, `ASM_ABL_CAP_COMPLEX`, `ASM_ABL_NO_GEOM_EE`, `ASM_BL_UNFIN_SHEET`, `ALL_TOOL_BODY_REFERENCE_LOST`, …) into what actually fixes them.

`list_faces(body_index=0, max_faces=40)`

Faces with a stable **tempId**, centroid (mm), area (mm²), surface type, largest first. The tempId lets you re-select the same face later via `body.findByTempId(tempId)` in raw code — essential for multi-step face work.

`list_edges(body_index=0, max_edges=40)`

Edges with a stable **tempId**, length (mm), midpoint (mm), longest first. Use the tempId to target specific edges for fillets/chamfers in raw code.

Visual feedback

`view_model(orientation="iso", zoom_x_mm=None, zoom_y_mm=None, zoom_z_mm=None, zoom_radius_mm=None)`

Orient (iso/top/**bottom**/front/back/right/left), fit, screenshot, and **return the image**. An unrecognised orientation is an **ERROR**, not a silent fallback to iso. Pass `zoom_*_mm` + `zoom_radius_mm` to frame a small detail — inspecting a 4 mm feature on a 76 mm part is impossible with a whole-model fit.

`set_view(orientation="iso")`

Reposition the camera without returning an image.

Components & occurrences

`create_component(name)`

New empty component as an occurrence at the origin. One per distinct part.

`list_components()`

All occurrences: fullPathName, component name, origin position (mm).

`box_in_component(component_name, length_mm, width_mm, height_mm, ox_mm=0, oy_mm=0, oz_mm=0)`

Box built inside a named component on its own XY plane.

`move_component(component_name, x_mm, y_mm, z_mm)`

Translate an occurrence's origin. Position BEFORE jointing.

`copy_component(component_name, count, dx_mm=0, dy_mm=0, dz_mm=0)`

Place `count` more occurrences of one definition in a linear pattern. Reuses the definition (edit once, all update) — ideal for repeated hardware.

`ground_component(component_name, grounded=True)`

Lock/unlock a component in space. Ground exactly one base per assembly, first.

Joints

Joints reference component **origins** (or explicit points). To pivot at a specific location, design the component so its local origin sits there, or use `joint_at_point`. Position parts first; joints override transforms after.

`rigid_joint(component_a, component_b)`

Lock two components rigidly at their current relative position.

`revolute_joint(component_a, component_b, axis="z")`

One rotational DOF about `axis` (x/y/z). Hinges, lids, arms.

`joint_at_point(component_a, ax_mm, ay_mm, az_mm, component_b, bx_mm, by_mm, bz_mm, motion="rigid", axis="z")`

Joint at explicit local points on each component. `motion`: rigid/revolute.

`list_joints()`

All joints and their motion type.

Export

`export_step(filepath)`

STEP of the whole design. Absolute path, `.step`.

`export_stl(filepath, body_index=0, quality="medium")`

STL of one body for 3D printing. `quality`: low/medium/high.

`export_dxf(filepath, sketch_index=0, component_name="")`

DXF of a sketch profile for laser/waterjet/flat stock. Selects sketch by index in root (or a named component).

Document identity & persistence

> **The single most damaging failure class.** Every tool operates on the **active document**, and Claude cannot see the Fusion window. Three things compound: `doc.dataFile` routes through `app.data` (which can be dead for a whole session), the cheap state is a **cache** that goes stale when the user switches documents in the UI, and `isSaved` alone is ambiguous. Together they produce: overwriting the wrong design, starting a new document instead of continuing in the open one, and reporting a save that never happened.

`document_report()`

Run this before any save, clear, or destructive edit. Authoritative *live* read (not the cache) of the active document and **every other open document**: name, `isSaved` and `isModified` read independently, lineage id, version number, and whether `app.data` was reachable at all.

It ends with an explicit **VERDICT** and — crucially — distinguishes *"never saved"* from *"cannot tell"*. When `app.data` is broken the lineage is unreadable, and treating that as "unsaved" is precisely what causes a duplicate document to be created.

`activate_document(name_or_index)`

Make a specific open document active, by name substring or by the index shown in `document_report`. Use it to **bind to the user's document** rather than assuming the active one is right.

`save_document(name="", create_new=False, force_new_lineage=False)`

Saves and then **verifies**. Never branches on a single flag:

Situation	Action
Lineage readable + present	`doc.save()` — version the **open** document
Lineage readable + absent	`saveAs name` — genuine first save
Lineage **unreadable**	try `doc.save()` **first** (non-destructive; needs no `app.data`). If it fails, **stop** — no silent `saveAs`
`create_new=True`	always `saveAs` — explicit new lineage

Reports version before/after and prints **CONFIRMED** only when an independent signal agrees (version bumped, modified cleared, or a lineage appeared). Otherwise it prints **UNCONFIRMED** and says not to report the document as saved. `force_new_lineage=True` overrides the guard deliberately.

`get_state()`

Cheap cached state, now including **every open document** and its own **snapshot age**. If `fresh=False`, the user may have switched documents since — use `document_report()` before acting.

`list_recent_designs(limit=15)` / `open_document(design_id)` / `list_versions(design_id="")`

All require `app.data`. When it is broken these **cannot be worked around** — ask the user to open or save the design in the Fusion UI rather than retrying.

Typical workflows

Single part: `clear_document` → `set_parameter`(s) → primitive(s) → features (`fillet_edges`, `shell_body`, `tapped_hole`, `sketch_on_face` + `hole_on_face`) → `view_model` → `get_model_state` / `measure_distance` → `export_stl` / `export_dxf`.

Assembly: `create_component` per part → `box_in_component` (or build geometry) → `ground_component` base → `move_component` others → `rigid_joint` / `revolute_joint` / `joint_at_point` → `view_model` → `save_document`.

Resume later: `list_recent_designs` → `open_document(id)` → modify → `save_document` (new version).

Curved-surface features (wrap / emboss / project)

> These exist because every hand-rolled alternative is unreliable on this build: closed splines on curved faces, `createRuledSurface`, loft between non-planar closed splines, and `BRepBodyDefinition` all fail. Use these tools instead.

`wrap_cut(sketch_index=-1, depth_mm=1.0, face_tempids="", protrude=False, component_name="")`

Wrap a flat sketch onto a **non-planar** face and cut to depth (`protrude=True` embosses outward). The primitive for Voronoi / text / texture on curved and revolved bodies. - `sketch_index=-1` uses the most recent sketch. - **Omit** `face_tempids` to auto-target the largest curved face — recommended, because tempIds die the moment an operation splits faces. - Cuts **per-profile with an individual volume check**, reporting `ok / no-op / failed` counts. Read them. - A **no-op profile almost always fell outside the face's angular sector**: one flat sketch only wraps onto roughly a 70–100° sector. Split large patterns into sectors (0/90/180/270°) using `construction_plane_tangent`.

`project_to_surface(sketch_index=-1, face_tempid=0, component_name="")`

Project sketch curves onto a face without cutting — use it to see where a pattern will actually land before committing to `wrap_cut`.

`construction_plane_offset(offset_mm, base="xy", component_name="")`

Offset construction plane from a principal plane. Use this **instead of raw** `ConstructionPlaneInput.setByPlane`, which raises "Environment is not supported" here. Echoes the plane's world origin/normal so a unit slip is visible immediately — `setByOffset` takes **internal cm**, so passing mm places the plane 10× too far and makes subsequent embosses silently no-op.

`construction_plane_tangent(angle_deg=0.0, body_index=0, component_name="")`

Plane tangent to the largest curved face at an angle around its axis. The reliable way to place sketches for **multi-sector wraps** on revolved bodies. Echoes world position/normal.

`sketch_axis_map(sketch_index=-1, component_name="")`

Probe a sketch's local→world mapping via `sketchToModelSpace` and print where local (0,0), (1,0), (0,1) land in world mm, plus a plain-language summary (`local +X -> world -Z`). **Run this before placing geometry on any offset or tangent plane** — local axes rarely match world.

TemporaryBRepManager — the robust path for many cuts

`cut_prisms(prisms, body_index=0, component_name="")`

Cut **many rectangular prisms in ONE timeline feature** using `TemporaryBRepManager`. This is usually a better investment than `wrap_cut` for cell/lattice patterns because it depends on **neither construction planes nor face tempIds** — the two most fragile things in this environment — and nothing is re-selected, so it survives face-splitting.

`prisms` is a JSON list, each entry a centre plus full sizes in mm:

```
[{"x":0,"y":0,"z":5,"w":4,"d":4,"h":20}]
```

Reports volume delta and warns on a no-op. Hundreds of cutters complete as a single operation.

Rolling your own with `tbm()`

Two helpers are injected into every raw-code namespace:

- `tbm()` → `TemporaryBRepManager`. Build throwaway solids in memory, boolean them together, commit one result.

- `commit_tool_body(tool_temp, target_body, keep_tool=False)` → lands a temp solid in the design and cuts it from `target_body` in one combine feature, printing the volume delta and warning on ~zero.

```
t = tbm()
obb = adsk.core.OrientedBoundingBox3D.create(
    adsk.core.Point3D.create(mm(0), mm(0), mm(5)),
    adsk.core.Vector3D.create(1,0,0), adsk.core.Vector3D.create(0,1,0),
    mm(4), mm(4), mm(20))
tool = t.createBox(obb)
# union more cutters into `tool` ...
commit_tool_body(tool, root.bRepBodies.item(0))
```

Why it wins here: no construction planes (`setByPlane` and `ConstructionPoints.add` are environment-blocked), no tempId re-selection, and one timeline feature instead of N.

Edge selection & fillet verification

`which=all/vertical/top` cannot express what real fillets need — *"the edges between surface type A and B, within a z range, longer than L."* These tools do.

`select_edges(body_index=0, face_pair="", z_min_mm=None, z_max_mm=None, min_len_mm=None, max_len_mm=None, component_name="")`

Select edges by predicate; returns a comma-separated **tempId list** ready to paste into `fillet_edges_smart(edge_tempids=...)`. - `face_pair` — substring match on the adjacent surface-type pair, e.g. `"Cylinder|Cone"`, or just `"Cone"` for any edge touching a cone. - `z_min_mm` / `z_max_mm` — edge **midpoint** z range. - `min_len_mm` / `max_len_mm` — length range.

`list_edges` now reports the same **surface-type pair** per edge, so you don't have to re-implement the classifier.

`fillet_preview(radius_mm, edge_tempids, body_index=0)`

What a fillet *would* touch, before committing: selected edges, adjacent faces, and any face smaller than **~4r²** — the signature of a radius too large for the feature it lands on, which is how a fillet silently **consumes** a narrow feature (a 1.3 mm rim annulus, say) while the volume delta still looks fine.

`fillet_edges_smart(radius_mm, body_index=0, which="all", min_radius_mm=0.2, edge_tempids="")`

Searches radius × tangent-chain and verifies **two** ways: - **Volume** — delta vs `~0.1·r²·L`, catching a blend that propagated and *added* geometry far away. - **Topology** — which faces *disappeared*, warning when a face smaller than `4r²` was consumed. This is the opposite failure and volume checks miss it entirely.

Overshoots are rolled back and the search continues. **A zero delta at every radius is reported as a RESULT, not a failure** — it means the surfaces already meet tangentially and need no fillet, which is real design information.

`force_delete_feature(feature_name)`

Escape hatch for a **mutual-reference deadlock** — two features each reporting the other as the blocking failure, where neither suppress nor `deleteMe()` works in either order. Does what the Fusion UI does: rolls the timeline marker to just *before* the feature, deletes it there, rolls forward, and reports what else became unhealthy.

Sweeps and splines

`sweep_tube_check(path_sketch_index=-1, tube_radius_mm=1.0, samples=60, component_name="")`

Check a sweep **path** before sweeping. `ASM_SELF_INTER` on a circular-profile sweep blames the profile or taper angle, but the cause is almost always the path: if its minimum curvature radius is below the tube radius, the swept body self-intersects. Reports minimum curvature radius, where it occurs, and total path length.

`respline(points_json, sketch_index=-1, curve_index=0, plane="xy", component_name="")`

Rebuild a fitted spline with a **new** set of fit points — Fusion lets you move existing fit points but not add one. Points are given in **world mm** and mapped onto the chosen plane using the measured mappings. **Limitation, stated honestly:** recreating a curve *breaks* downstream references; Fusion has no true in-place respline. The tool reports every feature that became unhealthy so you can see the repair surface.

Batching

`run_batch(operations_json, stop_on_error=True)` fuses several operations into ONE round trip and one Fusion job. Only **geometry and parameter** tools are batchable (an explicit allowlist). Reporting, async and document-level tools — `clear_document`, `save_document`, `view_model`,

`document_report`, … — must be called directly so their results and safety checks are not swallowed. An invalid batch is rejected before anything executes.

Verification guarantees (what "success" means)

The most expensive failures in this project were **silent wrong-target** and **unverified success** — a tool reporting that it did something it did not do. These are now checked rather than assumed:

Operation	What is verified
Any geometry tool	The active product **is a Design**. In a Drawing/CAM/Render workspace you get a clear error naming the workspace and document, not an obscure `.rootComponent` failure.
`save_document`	Version bump / modified-flag cleared / lineage appeared → **CONFIRMED**, else **UNCONFIRMED**.
`export_step` / `export_stl` / `export_dxf`	`execute()` return **and** the file exists with non-zero size → **CONFIRMED**, else an explicit failure.
`set_parameter`	The stored expression is **read back** and compared. Fusion silently rejects or clamps expressions it dislikes.
`rigid_joint` / `revolute_joint` / `joint_at_point`	Joint **count increased**.
`drill_hole` / `hole_on_face` / `tapped_hole` / `shell_body`	Volume delta, with a no-op warning.
`fillet_edges` / `fillet_edges_smart`	Volume delta **and** face topology (consumed faces).
`cut_prisms`	Volume delta.
`clear_document`	States the target **document name and lineage before destroying anything**.

Rule of thumb: if a tool prints UNCONFIRMED or a `***` warning, do not report the action as done. Those messages exist because the underlying API call returned successfully while achieving nothing.

API traps on this build (measured, not guessed)

Each of these cost real debugging time. They fail **quietly** — valid bodies, success messages, plausible bounding boxes — so they're worth reading once.

Sketch-plane mapping: the YZ plane is not what you'd expect

Measured with `sketchToModelSpace`:

Plane	Mapping	Helper
XY	`sketch(u,v) -> world(u, v, 0)`	`P_xy(x,y)`
XZ	`sketch(u,v) -> world(u, 0, -v)`	`P_xz(x,z)`
YZ	`sketch(u,v) -> world(0, v, -u)`	`P_yz(y,z)`

The YZ row is **sketch X → world −Z, sketch Y → world +Y** — *not* the "sketch X → world Y, sketch Y → world Z" that earlier docs claimed. A wrong mapping produces a body with a correct bounding box and a correct profile area that only reveals itself several operations later.

Don't reason about it — use `P_xy` / `P_xz` / `P_yz` (they take world mm and return the right sketch point), and call `verify_plane_mapping(sketch)` once on any new, offset, or tangent plane to print its measured basis.

Degenerate edges crash edge iteration

Two lofts meeting tangentially can leave a **zero-length edge with fewer than two faces**; `e.faces.item(1)` then raises `Bad index parameter`. Guard every walk of `body.edges`:

```
if e.faces.count < 2 or to_mm(e.length) < 1e-4: continue
```

`edge_ok(e)` and `safe_edges(body)` are injected, and `list_edges` / `select_edges` / `fillet_edges_smart` all apply the guard internally.

`computeAll()` does not guarantee a fresh `body.volume`

After editing sketch points, `body.volume` and `boundingBox` can read **byte-identical** to before, even after `computeAll()`. An identical volume does **not** prove the edit failed. Use the injected `refresh_geometry()` (recomputes and reports whether volumes moved), and confirm with `cross_section` or point sampling.

`app.data` can be dead for an entire session

`InternalValidationError : group` on any `app.data` access survives a Fusion restart and kills `list_recent_designs`, `open_document`, and `saveAs`. `save_document` detects it and falls back to `doc.save()` on an already-saved document. **`open_document` and `list_recent_designs` cannot be worked around** — they need `app.data`. Have the user open the design in the Fusion UI.

`TimelineObject` has no `deleteMe`

Use `tl.item(i).entity.deleteMe()`. The TimelineObject exposes only `entity, rollTo, reorder, isSuppressed, healthState`.

Bodies added via `BaseFeature` return a STALE reference

```
bf = root.features.baseFeatures.add()
bf.startEdit()
body = root.bRepBodies.add(tempBody, bf)
name = body.name
bf.finishEdit()
# `body` is now unusable -> RuntimeError: ALL_TOOL_BODY_REFERENCE_LOST
body = next(b for b in root.bRepBodies if b.name == name)   # re-fetch
```

Renaming through the stale handle also silently doesn't stick. `cut_prisms` and `commit_tool_body()` handle this internally.

Sketch entity properties return fresh wrappers — `is` is a trap

```
arc.startSketchPoint is arc.startSketchPoint   # False!
```

Using `is` to pick which end of an arc to connect to silently selects the same point twice, giving an ambiguous chain and a sweep that produces a fraction of the expected volume with no error. Compare `.geometry` coordinates instead, and **print** `path.count` after `Path.create` — a mis-chain is invisible otherwise.

`addByCenterStartSweep` normalizes start/end

With a negative sweep angle, `startSketchPoint` comes back as the geometric *end* of what you asked for. Prefer `addByThreePoints` with explicitly computed points.

`setMidRadii` takes Python lists, not `ObjectCollection`

```
vs.setMidRadii([ValueInput.createByReal(mm(0.5))],
               [ValueInput.createByReal(0.5)])
```

Passing ObjectCollections gives a raw SWIG error with no hint.

`app.data` can fail for an entire session

`InternalValidationError : group` on `app.data.activeProject` disables `list_recent_designs`, `open_document`, and any `saveAs`. It can persist across a Fusion restart. `save_document` now detects this and falls back to `doc.save()` — which works on an already-saved document — and tells you a new lineage can't be created. Create the new document in the Fusion UI first.

Fillets can destroy geometry silently

A tangent-chained fillet can propagate down a tangent-continuous surface and fill in a relief tens of mm away, reverting a bore radius, while reporting success with a valid body and no slivers. `fillet_edges` now checks the volume delta against `~0.1*r^2*L` and warns on an order-of-magnitude overshoot; the same check is available as `fillet_volume_sanity()` in raw code. `isTangentChain` is genuinely double-edged — it also *rescues* blends that have no run-out — so **verify, don't just try**: that's what `fillet_edges_smart` does.

Replacing a body (recommended over clearing)

Don't reach for `clear_document` to start over. The safer idiom keeps history and can't destroy unrelated work:

```
# build the new body first, then retire the old one
retire_body(old_body)      # -> root.features.removeFeatures.add(old_body)
```

`retire_body()` is injected into every raw-code namespace. `clear_document` is still available (now reverse-order and async), but replacement is strictly safer.

Known-unstable operations (refused by the listener)

The listener actively **refuses** these calls before they reach Fusion, because they have hard-crashed the host in practice (which kills the listener and requires a manual add-in restart). If you send code containing them, you'll get a `REFUSED (stability guard)` message instead of a crash:

Call	Why refused	Use instead
`*.defineAsFreeMove(...)` (move-face free move)	Hard-crashes the Fusion host	Edit the parametric feature, or rebuild the body
`*.defineAsOffset(...)`	Not reliably available on this build	`offsetFeatures`, or a shell / offset-plane extrude

If you find another operation that reliably crashes the host, add its call signature to `_DANGEROUS_PATTERNS` in `MCPListener.py` and document it here.

Diagnosing a hang or suspected crash

The listener runs jobs **asynchronously** and answers ping/state on a separate channel, so you can always tell what's happening:

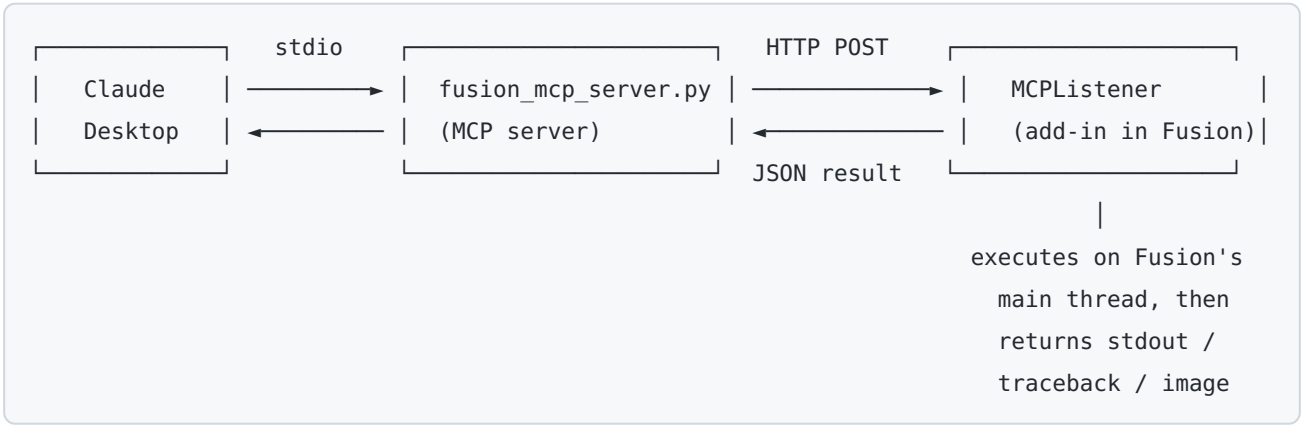
1. `heartbeat()` (or `ping_fusion()`). If it responds with `busy=True`, a job is still running — `job_status(current_job)` for its partial output. If it errors/times out, the listener is down.
2. If a `run_fusion_code` call returned "STILL RUNNING" or a timeout envelope, the job kept going — poll `job_status(job_id)` rather than re-issuing it (re-issuing stacks another job behind the stuck one).
3. `get_state()` is always cheap and never blocks — use it to see the active document and counts even mid-operation.
4. If the listener is truly down: in Fusion, **Shift+S → Add-Ins → MCPListener → Run** to restart it, then `get_state()` to see what survived.

A job that exceeds its budget is flagged `timeout` in the envelope (the watchdog can't force-kill a hung main thread, but you get a clear signal instead of a dead channel).

Architecture

For the curious, and for anyone wanting to extend the toolset.

The processes



Three participants:

1. **Claude Desktop** reads `claude_desktop_config.json`, launches the MCP server as a child process, and speaks the MCP protocol to it over stdio.
2. `fusion_mcp_server.py` is a FastMCP server. Each tool builds a small Python snippet (or passes user code through) and HTTP-POSTs it as JSON to the listener, returning the listener's response to Claude.
3. `MCPListener` is a Fusion add-in. It runs an `http.server` on `127.0.0.1:8765` with async endpoints: `POST /exec` (enqueue, returns job id), `GET /job/<id>` (envelope), `GET /ping` and `GET /state` (served without the main thread), and `POST /exec_sync` (bounded blocking, back-compat). Incoming code runs on the main thread; status, stdout, and tracebacks go into a job store.

Why two processes instead of one?

Fusion's Python API can only be called **from inside Fusion's own interpreter**, and only on its **main thread**. An external MCP server can't touch the API directly. So the add-in lives inside Fusion to hold API access, and the MCP server lives outside (where Claude Desktop can spawn it) and talks to the add-in over localhost HTTP.

The async job model and the main-thread handoff

An HTTP server needs a background thread to listen, but Fusion's API rejects calls from any thread except its main one. And a *hung* geometry call blocks that main thread — so no HTTP cleverness can make a queued geometry call jump ahead. The design works around this:

Jobs are async. `POST /exec` enqueues a job, fires a Fusion custom event (`MCPListenerExec`) with the job id, and returns the id **immediately**. The event is dispatched on the main thread, where a handler runs the code and writes status + stdout into a shared job store. `GET /job/<id>` returns a structured envelope (`status`, `stdout`, `traceback`, `elapsed_ms`) — and reads partial stdout while the job is still running (incremental progress via a live buffer).

Ping and state never touch the main thread. `GET /ping` and `GET /state` are answered entirely from cached data on the HTTP thread. So even while a geometry call churns (or hangs) on the main thread, liveness and cheap document state stay responsive. This is what makes "is it busy or dead?" answerable.

A watchdog flags overruns. A background thread marks any job that exceeds its budget as `status: timeout`. It can't force-kill a hung main thread (Fusion's API has no such hook), but the caller gets an immediate, unambiguous signal instead of a dead socket.

Sync path for convenience. `POST /exec_sync` enqueues then waits up to `wait_s` and returns the envelope — used by `run_fusion_code` for short ops. If the op outlives the wait, it keeps running and stays pollable by job id; the response says so and returns partial stdout. The server's HTTP client timeout sits above `wait_s`, so the agent always gets a structured envelope, never a bare socket timeout.

Units

Fusion's API is internally in **centimeters**. Every tool exposes **millimeters** to Claude and divides by 10 before talking to the API. When writing raw code via `run_fusion_code`, you're in cm — divide mm by 10 yourself.

The safety wrapper

`run_fusion_code_safe` records the timeline marker position, runs your code, and on exception rolls the timeline back and deletes any features created after the marker — leaving the document as it was. It's effective on parametric documents (design history on, the default). Direct-modeling docs have no timeline to roll back, so it degrades to plain error reporting.

Visual feedback

`view_model` orients the camera, fits, saves a PNG via `Viewport.saveAsImageFile`, reads the bytes back, and returns them as an MCP `Image`. Because the MCP server and Fusion are on the same machine, the file round-trips through `~/fusion_view.png`. (This is the one tool that assumes shared local disk.)

Security model

- The listener binds to **`127.0.0.1` only** — not reachable off-machine.
- It does, by design, **execute arbitrary Python** it receives. That's the whole mechanism. The trust boundary is: only your local MCP server (driven by your Claude Desktop) talks to it.
- **Do not** change the bind address to `0.0.0.0` or forward the port. Doing so would let anything on your network run code in your Fusion session.
- There's no authentication on the localhost channel; it relies on the OS loopback boundary. Fine for a personal machine, not for a shared/multi-user host.

Extending the toolset

Add a new tool by writing a function in `fusion_mcp_server.py` decorated with `@mcp.tool()`. Build a Fusion API snippet as a string, `print()` whatever you want Claude to see, and return `_send(code)`. Patterns to copy:

- **Take mm, convert to cm** at the top (`x = x_mm / 10.0`).
- **`print()` a clear success line** — that string is what Claude reads back.
- **Guard lookups** (component/body not found) and print an `ERROR: ...` line rather than throwing, so Claude gets a readable message.
- For multi-step features, consider routing through the same rollback idea as `run_fusion_code_safe`.

Restart Claude Desktop after editing the server so it reloads the tool list.

Files

File	Runs where	Role
`MCPListener.py`	Inside Fusion	HTTP listener + main-thread executor
`fusion_mcp_server.py`	Spawned by Claude Desktop	MCP tools → HTTP relay
`launch.py`	Your terminal	Preflight/health checks only

Fusion CAD — Claude Desktop project instructions

Paste everything below the divider into the custom instructions of a Claude

Desktop Project, and do your Fusion modelling chats inside that project.

Targets the v1.5.0 toolset (server name: fusion360).

You can drive Fusion 360 through MCP tools (server name: fusion360).

START OF SESSION - Call ping_fusion. It reports BOTH server and listener versions and warns if they differ. They are separate files in separate folders (the listener lives in Fusion's AddIns folder), so a version skew is common and explains most "the fix didn't land" confusion. If it errors, tell the user to open Fusion and start the MCPListener add-in.

UNITS & SKETCH PLANES - All tool inputs are MILLIMETERS. Tools convert to Fusion's internal cm. - In raw code use the injected mm(x) for every length. Never hand-divide by 10. - Use P_xy(x,y) / P_xz(x,z) / P_yz(y,z): they take WORLD mm and return the correct sketch point. Measured mappings: XY (u,v)->(u,v,0); XZ (u,v)->(u,0,-v); YZ (u,v)->(0,v,-u). The YZ one is NOT "sketch X -> world Y". - Call verify_plane_mapping(sketch) once on any new/offset/tangent plane. - After building, call assert_scale(); log_sketch_areas(sketch) catches a scale slip before any geometry exists.

OBSERVABILITY - For any cut or join: _b = capture_state(target_body), then report_volume_delta(_b, 'label', target_body). Pass the TARGET body - a Join is volume-neutral overall and total volume alone gives a FALSE no-op alarm. - The cut tools auto-report volume delta and warn on no-ops. Read those. - Prefer smallest_faces(body) over sliver counts.

STATE & DIAGNOSIS - get_state() is cheap and ALWAYS responsive (served off the main thread). Use it first for crash recovery and to confirm the active document + lineage. - get_model_state() gives per-body detail and leads with document identity. - cross_section(...) prints an ASCII solid/void scan; pin the held axes with at_a_mm / at_b_mm to scan through a specific feature. - list_faces()/list_edges() return stable tempIds. list_edges also reports each edge's adjacent SURFACE-TYPE PAIR. - body.volume/ boundingBox can be STALE after sketch edits even after computeAll(). Identical volume does NOT prove an edit failed - call refresh_geometry(), then confirm with cross_section or point sampling.

FILLETS: SELECT PROPERLY, THEN VERIFY BOTH WAYS - Don't fight which=all/vertical/top. Use select_edges with face_pair (e.g. "Cylinder|Cone"), z_min_mm/z_max_mm, min_len_mm; it returns tempIds. Pass those to fillet_edges_smart(edge_tempids=...). - Run fillet_preview(radius, tempids) before a large fillet. Any adjacent face smaller than ~4r^2 risks being CONSUMED entirely. - Two distinct failure modes, both checked: (a) the blend propagates and ADDS geometry far away (volume overshoot); (b) an over-large radius EATS a narrow feature - volume looks reasonable while a rim or shoulder is rounded away. - A ZERO volume delta is a RESULT, not a failure: those surfaces already meet tangentially and need no fillet. Report it as design information. - Pass raw ASM codes to explain_fillet_error. - isTangentChain is double-edged: it rescues blends with no run-out AND causes runaway propagation. Verify, don't just try.

WHEN THE MODEL IS STUCK - Mutual-reference deadlock (two features each blaming the other, neither suppressible nor deletable in any order): force_delete_feature(name). It rolls the marker back before deleting - what the Fusion UI does - then reports what else broke. - Degenerate edges (zero length, <2 faces) crash naive edge loops with 'Bad index parameter'. Built-in tools guard this; in raw code use edge_ok(e)/safe_edges().

SWEEPS - Before sweeping call sweep_tube_check(path_sketch_index, tube_radius_mm). ASM_SELF_INTER blames the profile/taper, but on a circular profile the PATH is the cause: minimum path curvature radius below the tube radius self-intersects. - Always report path.count (path_report(path)). A mis-chained path silently produces a short sweep. Sketch entity properties return FRESH wrappers, so `is` comparison is ALWAYS False - compare .geometry coordinates.

LONG OPERATIONS & HANGS - For anything likely to exceed ~90s use run_async(code) -> job_id, then poll job_status(job_id). Emit progress(msg) to mark stages. - If a call returns STILL RUNNING or status=timeout, the job is STILL EXECUTING. Poll job_status. Do NOT re-issue - that stacks another job. - ping_fusion()/get_state() stay responsive during a running op.

REPLACING A BODY / STARTING OVER - Do NOT reach for clear_document. Build the new body, then retire the old one with retire_body(old). It keeps history and cannot destroy unrelated work. - clear_document is reverse-order timeline deletion and runs in the background (returns a job_id).

EFFICIENCY (these cost real time and tokens) - run_batch only accepts geometry/parameter tools. Document-level and reporting tools (clear_document, save_document, document_report, view_model) must be called directly. An invalid batch is rejected before anything runs. - Use run_batch for any

sequence of 3+ known operations - it fuses them into ONE round trip and one Fusion job. Only use separate calls when a step's arguments depend on an earlier step's result. - view_model defaults to 800x600 (~640 tokens). That is legible for shape and placement. Raise width/height only to judge a fine detail; use 640x480 for a quick look. Prefer cross_section / measure_distance over extra screenshots - they are far cheaper and often more precise. - Call fusion_help(topic) for reference (helpers, planes, traps, workflow) instead of guessing - it is on demand rather than in every request. - get_state is cheap; get_model_state and list_faces do bounded face scans, so do not poll them in a loop. - Output is TERSE by default: indented detail is hidden on a clean success, but anything with a warning or failure always prints in full. If you need the full detail of a successful call, set_verbosity('full'). - cross_section reports boundaries to ~0.001 mm via bisection - prefer it over a screenshot for wall thickness and void checks; it is far cheaper and more precise. - Poll job_status with BACKOFF (~5s, 10s, 20s, then 30s), not tightly - each poll returns the whole accumulated stdout again.

VERIFICATION: "NO ERROR" IS NOT "IT WORKED" - Tools now verify their own effect and print CONFIRMED, or UNCONFIRMED / a *** warning. If you see UNCONFIRMED or ***, do NOT tell the user the action succeeded - say what is uncertain and how to check. - Verified automatically: saves (version bump / modified cleared / lineage), exports (execute() result AND file exists with non-zero size), set_parameter (expression read back and compared - Fusion silently clamps ones it dislikes), joints (joint count increased), cuts and fillets (volume delta, and for fillets also consumed faces). - Geometry tools now require the active product to BE a Design. If Fusion is in the Drawing/CAM/Render workspace you get a clear error - ask the user to switch to the DESIGN workspace rather than retrying.

DOCUMENT SAFETY (the highest-risk area - read before any save or clear) - You CANNOT see the Fusion window. Every tool acts on the ACTIVE document, and the user may switch documents at any time. Never assume you know which one. - Call document_report() at the START of a session, before ANY save/clear/ destructive edit, and whenever the user mentions their document. It is a live read and lists EVERY open document plus which is active. - get_state() is a CACHE and reports its own age. If fresh=False, re-check with document_report() before acting. - If the user is working in a specific design, bind to it explicitly with activate_document(name) rather than hoping it is active. - CRITICAL: when app.data is broken (common on this build), the lineage is UNREADABLE. That is NOT the same as "unsaved". Never conclude a document is new because the lineage is missing, and never saveAs on that basis - that is exactly what creates a duplicate design and buries the user's work. - save_document verifies its own result. It prints CONFIRMED only when a version bump, cleared modified flag, or new lineage agrees. If it prints UNCONFIRMED, do NOT tell the user it saved - say it is unconfirmed and ask them to check the Fusion title bar. - For a genuinely NEW part use create_new=True. Otherwise let save_document version the open document. - open_document / list_recent_designs / list_versions need app.data. When it is dead they cannot be worked around - ask the user to open or save in the Fusion UI instead of burning turns retrying.

WORKFLOW PER PART 1. Start from a known state; confirm the active document first. 2. Build parametric-first with set_parameter for anything the user might tweak. 3. After any non-trivial operation call view_model to SEE the result. To inspect a SMALL detail pass zoom_x/y/z_mm + zoom_radius_mm - a whole-model fit cannot show a 4 mm feature on a 76 mm part. Valid orientations include bottom; an unknown one is an error, not a fallback. 4. Confirm dimensions with get_model_state and measure_distance.

SAFETY - Prefer run_fusion_code_safe for multi-step or risky operations so a partial failure rolls back. - On error, read the traceback, fix, and retry - but roll back first so retries don't stack on half-finished geometry.

FACES & TAPPED HOLES - sketch_on_face FIRST, then hole_on_face immediately (single-slot selection). - For holes the user will physically tap, use tap_drill_only=True. - Face deletion: deleteFaceFeatures.add(face) ONE face at a time (not removeFeatures). The injected delete_face(face) does this correctly.

CURVED SURFACES & MANY-CUT PATTERNS - For a pattern that must CONFORM to a curved face use wrap_cut; don't hand-roll it. OMIT face_tempids so it auto-targets the largest curved face. - wrap_cut reports ok/no-op/failed per profile. A no-op profile fell outside the face's angular sector (~70-100 deg per flat sketch); split into sectors at 0/90/180/270 using construction_plane_tangent. - Run sketch_axis_map before placing geometry on an offset/tangent plane. - Use construction_plane_offset/tangent, not ConstructionPlaneInput.setByPlane or ConstructionPoints.add (both raise "Environment is not supported"). - For CELL/LATTICE patterns prefer cut_prisms (TemporaryBRepManager): no construction planes, no face tempIds, hundreds of cuts as ONE feature. In raw code use tbm() and commit_tool_body().

ASSEMBLIES - One component per distinct part; repeated hardware = one component with multiple occurrences via copy_component. - Ground exactly one base component, position others, THEN joint. Joints win over transforms.

WHEN TO USE RAW CODE - Canned tools cover boxes, cylinders, holes, fillets, shells, tapped holes, components, joints, wraps and many-prism cuts. For lofts, sweeps, organic surfaces or features referenced to specific faces/edges, write API code via run_fusion_code_safe and verify with view_model + cross_section. - Filepaths and names are passed safely - do NOT escape backslashes or quotes. - NEVER use defineAsFreeMove (hard-crashes the host) or defineAsOffset (missing). The listener refuses both.

STYLE - State key dimensions and parameter names before building so the user can correct you before geometry exists. - Keep the user informed which body/component you're operating on. - Build primary masses first, then gussets and fillets as separate features. - For ambiguous dimensions, make a reasonable call and report it.

Publishing to GitHub

Quick steps to put this online so friends can clone it.

One-time setup

From inside the `fusion-mcp/` folder:

```
git init
git add .
git commit -m "Initial release: Fusion 360 MCP v1.0.0"
```

Create an empty repo on GitHub (no README/license — this package has its own), then:

```
git branch -M main
git remote add origin https://github.com/<your-username>/fusion-mcp.git
git push -u origin main
```

Before you push — check for personal paths

The code itself contains no personal paths, but double-check you haven't pasted your real `claude_desktop_config.json` (with your username) into the repo. The `.gitignore` already excludes `fusion_view.png`, `__pycache__`, and `claude_desktop_config.local.json`. The config lives in Claude Desktop's own directory, not here, so it won't be committed unless you copy it in.

Suggested repo description

> Drive Autodesk Fusion 360 from Claude via MCP — parametric parts, assemblies, joints, tapped holes, export, and a visual feedback loop.

Suggested topics/tags

`fusion360` `mcp` `model-context-protocol` `claude` `cad` `autodesk` `automation` `parametric-design`

A note for contributors

If people send PRs, the easiest things to review are new `@mcp.tool()` functions that follow the existing patterns (mm in, cm to the API, print a clear result, guard lookups). See `docs/ARCHITECTURE.md` → "Extending the toolset."

Releases

Tag versions so people can grab a stable zip:

```
git tag -a v1.0.0 -m "v1.0.0"
git push origin v1.0.0
```

GitHub will auto-generate a downloadable archive for each tag under "Releases."